

BORDER OPTICS — POORI RESEARCH SAMJHO

PDF 15 of 22 — Robustness and Advanced Stats: Placebo Tests
`pretreatment_placebo_test.py`, `dw_pretreatment_placebo_test.py`

Sakshi ke liye, Sakshi ki apni research par. PDF 12 mein humne DiD (Difference-in-Differences) seekha tha, aur uske saath ek zaroori, hidden assumption bhi aayi thi — 'parallel trends': ki agar VVP-I na hoti, toh treated aur control villages waise hi, ek jaisi rafta se badalte rehte. Us assumption ko seedha test nahi kiya ja sakta, kyunki hum history dobara nahi chala sakte. Lekin is PDF ki dono files ek chaalaak tareeka dhoondhti hain — ek aisa time-period dhoondho jahan VVP-I ka asar ho hi nahi sakta tha, aur wahan wahi DiD test chalao. Agar wahan bhi null milta hai, toh parallel-trends assumption par bharosa karna zyada justified ban jaata hai.

Is PDF mein ek genuinely interesting, deeply honest moment bhi hai — jahan is poore project mein pehli baar ek result 'borderline' nikalta hai, aur script ka apna automatic pass/fail check use galat tarike se 'PASS' keh deta, agar usse blindly follow kiya jaata. Dekhte hain ki us moment ko kaise handle kiya gaya.

Shuru Karne Se Pehle — 'Placebo Test' Ka Matlab Kya Hai, Aur Ye Zaroori Kyun Hai

PDF 12 mein `did_model.py` seekhte waqt, humne DiD (Difference-in-Differences) ka poora logic samjha tha: treated villages ka before-se-after change nikaalo, control villages ka before-se-after change nikaalo, aur dono changes ka farak hi asli, causal effect hai — kyunki jo bhi cheez treated aur control, dono ko affect karti hai (jaise poore region mein ek general economic trend, ya climate pattern), wo dono taraf se cancel ho jaati hai, subtraction mein.

Lekin ye poora tareeka ek chhupi hui shart par tika hua hai, jise 'parallel trends assumption' kehte hain: ye maan liya jaata hai ki agar VVP-I kabhi sanction hi na hoti, toh treated aur control villages, dono, waise hi, ek jaise rafta (parallel trends) se badalte rehte — jo farak humein dikh raha hai wo sirf treatment ki wajah se hai, kisi pehle-se-maujood, alag-alag trajectory ki wajah se nahi. Problem ye hai ki is assumption ko seedha, directly test nahi kiya ja sakta — hum history ko dobara nahi chala sakte ye dekhne ke liye ki 'agar VVP-I na hoti toh kya hota.'

Lekin ek chaalaak, indirect tareeka available hai: agar treated aur control villages sach mein parallel trends follow karte hain, toh unhe kisi bhi aise period mein bhi similar tarah se badalna chahiye jahan treatment ka asar possible hi nahi tha. VVP-I ko February 2023 mein sanction kiya gaya tha — matlab 2019 se 2021 tak ka period (dono saal treatment se pehle) mein koi bhi asar hona physically impossible hai. Agar is 'placebo' period mein bhi treated aur control villages mein koi significant farak nahi milta — waisi hi DiD test lagane par — toh ye is study ke asli, 2021-2025 wale DiD test ke liye ek genuine, real evidence hai (proof nahi, evidence) ki dono groups comparable hain.

EXAMPLE — Ek Doctor Ki Placebo-Pill Wali Analogy

Medicine testing mein 'placebo' ka matlab hota hai ek aisi pill jisme koi asli dawa nahi hoti — sirf sugar. Agar researchers dekhna chahte hain ki ek nayi dawa sach mein kaam karti hai ya nahi, wo ek group ko asli dawa dete hain aur doosre ko placebo, phir dono groups ka result compare karte hain. Yahan bhi bilkul wahi spirit hai, bas thoda ulta application: hum 2019-2021 ko ek 'placebo period' ki tarah use kar rahe hain — is period mein 'treatment' (VVP-I) exist hi nahi karti thi, isliye agar hum wahi test yahan bhi chalayein aur wo bhi null aaye, toh ye confirm karta hai ki humara testing-tareeka khud kisi random, unrelated farak ko 'false positive' ki tarah nahi pakad raha — jab treatment maujood hi nahi thi, tab bhi ye test sahi tarah 'kuch nahi mila' bata raha hai.

Is family mein do files hain: `pretreatment_placebo_test.py` NDBI aur night-lights ke liye ye placebo test karta hai — bilkul wahi do proxies jo H1 (PDF 11) aur H4 (PDF 12) mein

use hui thi. `dw_pretreatment_placebo_test.py` wahi test, Dynamic World naam ke ek teesre, alag satellite-based proxy ke liye karta hai — jiski poori kahani PDF 18 (Triangulation) mein milegi, lekin yahan itna jaanna kaafi hai ki Dynamic World bhi 'built-up area kitni hai' predict karne ki koshish karta hai, bas ek alag algorithm (Google ka land-cover classifier) se, NDBI ke seedhe do-band formula se nahi.

Part A — `pretreatment_placebo_test.py`

Docstring Aur Imports

```
"""
Parallel-pre-trends placebo test (Section 7.5): checks 2019-to-2021 change,
treated vs control, both pre-treatment (VVP-I wasn't sanctioned till Feb 2023).
Want this to come back null - that's what supports the real DiD's pre-trends
assumption. Same spec as did_model.py's H4 (district FE, cluster-robust SE).

Needs extract_pretreatment_baseline.py run for both groups/windows first.

Usage:
    python3 src/analysis/pretreatment_placebo_test.py --window full_year
    python3 src/analysis/pretreatment_placebo_test.py --window summer
"""

import argparse
import json

import pandas as pd
import statsmodels.formula.api as smf
from scipy import stats
```

Purpose: Docstring khud is file ka poora maqsad ek jagah bata deta hai: 2019-se-2021 ka change, treated vs control, dono hi periods treatment se pehle ke — isliye result null aana hi 'good news' hai, kyunki ye asli DiD ki parallel-trends assumption ko support karta hai. 'Same spec as `did_model.py`'s H4' likha hona zaroori hai — matlab is file ka regression-formula wahi hai jo PDF 12 mein seekha tha, sirf ek chhota structural farak ke saath (aage dekhenge). Ek dependency bhi note hai: `extract_pretreatment_baseline.py` pehle chalni chahiye, jo 2019 ka NDBI/lights data nikaalti hai (ye acquisition-side script hai, is PDF ka hissa nahi, sirf iska input taiyaar karti hai). **Syntax:** `import argparse, json, pandas as pd` — is poore series mein bahut baar dekhe ja chuke hain (command-line arguments, JSON save/load, dataframes). `import statsmodels.formula.api as smf, from scipy import stats` — PDF 12 mein pehli baar dekhe the (R-style formula regression, Mann-Whitney jaisa statistical test) — is file mein dono ek saath use honge.

Paths — Teen Dictionaries

```

PRETREATMENT_PATHS = {
    ("treated", "full_year"):
        "data/processed/border_optics_treated_pretreatment_fullyear.csv",
    ("treated", "summer"):
        "data/processed/border_optics_treated_pretreatment_summer.csv",
    ("control", "full_year"):
        "data/processed/border_optics_control_pretreatment_fullyear.csv",
    ("control", "summer"):
        "data/processed/border_optics_control_pretreatment_summer.csv",
}
EXISTING_2021_PATHS = {
    ("treated", "full_year"): "data/processed/border_optics_village_results_analyzed.csv",
    ("treated", "summer"):
        "data/processed/border_optics_village_results_summer_analyzed.csv",
    ("control", "full_year"): "data/processed/border_optics_control_results.csv",
    ("control", "summer"): "data/processed/border_optics_control_results_summer.csv",
}
SUMMARY_OUT = {
    "full_year": "outputs/pretreatment_placebo_summary_fullyear.json",
    "summer": "outputs/pretreatment_placebo_summary_summer.json",
}

```

Purpose: PRETREATMENT_PATHS batata hai kahan se 2019 ka data milega — chaar files, har group (treated/control) aur har window (full_year/summer) ka apna alag file. EXISTING_2021_PATHS ek nayi, interesting cheez hai — ye wahi files hain jo pehle se hi is project mein exist karti hain (jaise border_optics_village_results_analyzed.csv, jo PDF 11 ke analyze_results.py ne bhi use ki thi) — is file ka 2021 ka data inhi purani, already-computed files se liya jaata hai, dobara extract nahi kiya jaata. Ye ek achi design choice hai: jo data pehle se available hai, use dobara mangwane ke bajaye, seedha reuse kar liya. Syntax: Dictionary ki key is baar ek tuple hai — ("treated", "full_year") — matlab ek single value ke bajaye, do values ko jodkar ek key banayi gayi hai. Ye pattern tab useful hota hai jab do alag dimensions (group aur window) ka combination ek unique file decide karta hai. Dictionary lookup waisa hi rehta hai — PRETREATMENT_PATHS[("treated", "full_year")] seedha us specific file ka path de dega.

build_group() — Treated Ka Simple Join, Control Ka Coordinate-Based Join

```

def build_group(group, window):
    """Returns one row per village: village_id, district, treatment,
    ndbi_2019, ndbi_2021, ndbi_placebo_change (=2021-2019), same for lights."""
    baseline = pd.read_csv(PRETREATMENT_PATHS[(group, window)])
    existing = pd.read_csv(EXISTING_2021_PATHS[(group, window)])
    if group == "treated":
        existing = existing[existing["is_core_sample"] == True]
        # treated village_id is stable across runs, safe to join on

```

```

baseline = baseline[["village_id", "ndbi_2019", "lights_2019"]]
existing = existing[["village_id", "district", "ndbi_before", "lights_before"]].rename(
    columns={"ndbi_before": "ndbi_2021", "lights_before": "lights_2021"}
)
merged = existing.merge(baseline, on="village_id", how="inner")

```

Purpose: Ye function ek group (treated ya control) ke liye 2019 aur 2021, dono saalon ka data ek hi dataframe mein jodta hai. Docstring khud bata deta hai ki final output kaisa dikhega — har row ek village, uska 2019 aur 2021 ka NDBI/lights, aur unka farak (placebo_change). Treated group ke liye tareeka seedha hai: `is_core_sample==True` wale rows hi rakho (PDF 11 se familiar filter), baseline (2019) se sirf zaroori columns lo, existing (2021, jo `ndbi_before/lights_before` naam se already stored tha `did_model.py` ki bhasha mein) ko rename karke `ndbi_2021/lights_2021` bana do, aur `village_id` par seedha merge kar do — kyunki treated `village_id` kabhi badalta nahi, is join ke liye ye ek safe, reliable key hai. Syntax: `.merge(baseline, on="village_id", how="inner")` — pandas ka join operation, SQL ke JOIN jaisa hi — dono dataframes ki rows ko `village_id` ke match hone par jod deta hai; `how="inner"` ka matlab sirf wahi rows rakho jo dono taraf maujood hain, kisi ek taraf missing wali rows drop ho jaayengi. `.rename(columns={...})` — column ka naam badalna, dictionary format mein purana-naam:naya-naam. `== True` ek boolean column ko filter karne ka tareeka, sirf True wali rows rakhna.

```

else:
    # control village_id gets reassigned fresh every time
    # select_control_villages.py regenerates the list, so it is NOT a safe
    # join key here -- two different control-list vintages (this baseline's
    # vs. the current EXISTING_2021_PATHS file) can each have a row with
    # village_id=5 that refers to a completely different physical village.
    # Join on rounded coordinates instead, which are stable per village.
    baseline = baseline.copy()
    existing = existing.copy()
    baseline["_coord_key"] = list(zip(baseline["latitude"].round(6),
    baseline["longitude"].round(6)))
    existing["_coord_key"] = list(zip(existing["latitude"].round(6),
    existing["longitude"].round(6)))
    baseline = baseline[["_coord_key", "ndbi_2019", "lights_2019"]]
    existing = existing[["_coord_key", "village_id", "district", "ndbi_before",
    "lights_before"]].rename(
        columns={"ndbi_before": "ndbi_2021", "lights_before": "lights_2021"}
    )
    merged = existing.merge(baseline, on="_coord_key",
    how="inner").drop(columns="_coord_key")
    n_dropped = len(existing) - len(merged)
    if n_dropped:
        print(f" NOTE: {n_dropped} control village(s) in {EXISTING_2021_PATHS[(group,
    window)]} ")

```

```

f"have no matching coordinate in {PRETREATMENT_PATHS[(group, window)]}
-- the 2019 "
f"baseline is for a different control-list vintage and should be re-extracted "
f"(extract_pretreatment_baseline.py --group control) before trusting this
result.")

```

Purpose: Control villages ke liye `village_id` par join karna khatarnak hai, aur ye comment khud iski wajah bilkul saaf-saaf bataata hai — control village-list har baar `select_control_villages.py` chalane par dobara ban sakti hai, aur har baar naye sirey se numbered ho sakti hai (`village_id=5` pehli baar kisi aur physical gaon ko point kar sakta hai, doosri baar kisi aur ko). Isliye yahan ek zyada stable identifier chahiye — village ke apne coordinates (latitude/longitude), jo kabhi nahi badalte, chahe list kitni baar regenerate ho. Ek '`_coord_key`' naam ka naya, temporary column banaya jaata hai jismein har row ka (latitude, longitude) pair, rounded to 6 decimal places, ek single value ki tarah store hota hai — aur usi par join hoti hai. `n_dropped` ek defensive check hai: agar kisi row ka koi match hi nahi milta (matlab 2019 wali file kisi purani, alag control-list vintage ki hai), toh ek loud warning print hoti hai, chup-chaap kuch drop nahi hota. **Syntax:** `df.copy()` — dataframe ki ek independent copy banana, taaki original dataframe accidentally modify na ho. `list(zip(col1, col2))` — `zip()` do columns ko row-by-row pair karta hai (jaise (lat1,lon1), (lat2,lon2), ...), `list()` usse ek Python list mein badal deta hai — ye 'tuple ko dictionary/join-key ki tarah use karna' ka wahi pattern hai jo PDF 14 mein `district_idx` dictionary mein bhi dekha tha, bas yahan ek column ki jagah poori ek naya column ban raha hai. `.round(6)` — decimal ko 6 places tak round karna, taaki floating-point ke chhote-chhote farak (jaise 0.000001 ka antar, jo actually same hi location hai) match karne se na rok sakein. `.drop(columns="x")` — ek column hataana, yahan temporary `_coord_key` ko merge ke baad hata diya gaya, kyunki aage ki analysis mein iski zaroorat nahi. `len(df)` — rows ki ginti; do lengths ka farak (`n_dropped`) batata hai kitni rows match nahi hui.

EXAMPLE — Ye Wahi Bug Hai Jo Entry 37 Ne Pehli Baar Pakda Tha

Ye coordinate-based join koi naya invention nahi hai is file ke liye — PDF 14 mein humne dekha tha ki Development Log ki Entry 37 mein exactly yehi problem (control `village_id` ka har regeneration mein badal jaana) pehli baar pakda gaya tha, jab `extract_dynamicworld_built.py` aur `extract_sar_backscatter.py` control-side data ko purani, stale checkpoint se load kar rahi thi. Us waqt jaanch karte hue, Entry 37 ne yahi pattern do aur jagah bhi dhoondh nikaala tha — `pretreatment_placebo_test.py` aur (tab tak sirf likhi gayi, chalayi nahi gayi) `dw_pretreatment_placebo_test.py` dono treated aur control data ko `village_id` se jod rahi thi, jo treated ke liye toh theek tha (uska id stable hai), lekin control ke liye risky — agar kabhi ek naye control-list vintage ke baad ye script chalayi jaati bina fix kiye, toh chup-chaap galat villages jud jaate. Ye entry mein khud likha hai: 'this hadn't corrupted anything yet... but running either script again without first re-extracting a fresh 2019 baseline against the current list would have silently paired up the wrong villages.' Dono scripts isi Entry

37 mein coordinate-based join se fix ki gayi — jo code hum abhi upar padh rahe hain, wahi fix hai.

placebo_did() — did_model.py Jaisa Formula, Bina 'post' Term Ke

```
def placebo_did(panel, outcome_col):
    # same spec as did_model.py's H4, but outcome is already the 2019-2021
    # change so there's no 'post' term - just treated vs control on that change
    d = panel.dropna(subset=[outcome_col]).copy()
    model = smf.ols(f"{outcome_col} ~ treatment + C(district)", data=d).fit(
        cov_type="cluster", cov_kwds={"groups": d["district"]}
    )
    coef, se, p = model.params["treatment"], model.bse["treatment"],
    model.pvalues["treatment"]
    ci = model.conf_int().loc["treatment"]
    u_stat, mw_p = stats.mannwhitneyu(
        d[d.treatment == 1][outcome_col], d[d.treatment == 0][outcome_col]
    )
    return {
        "n_obs": len(d),
        "n_treated": int((d.treatment == 1).sum()),
        "n_control": int((d.treatment == 0).sum()),
        "placebo_did_coef": float(coef), "placebo_did_se": float(se), "placebo_did_p":
        float(p),
        "placebo_did_ci_lo": float(ci[0]), "placebo_did_ci_hi": float(ci[1]),
        "mann_whitney_p": float(mw_p),
    }
```

Purpose: Comment khud is function ka sabse important structural point bata deta hai: did_model.py (PDF 12) mein formula tha "value ~ treatment + post + did_term + C(district)" — kyunki wahan panel wide-to-long reshape hoke, har village ke do rows the (before aur after), aur did_term (treatment*post interaction) hi asli causal estimate tha. Yahan aisi zaroorat nahi hai, kyunki ndbi_placebo_change column mein 2021-minus-2019 ka farak pehle se hi compute ho chuka hai (build_group() mein) — har village ka ek hi row hai, jismein uska 'change' already stored hai. Isliye ab sirf ek seedha sawaal poochna hai: 'treated villages ka ye change, control villages ke change se alag hai kya?' — jiske liye sirf treatment column ka coefficient chahiye, koi post ya interaction term nahi. District fixed effects (C(district)) aur cluster-robust SE (cov_type="cluster") wahi rahte hain — PDF 12 mein seekha gaya wahi tareeka. Ek extra check bhi hai: stats.mannwhitneyu() — yahi test jo PDF 12 mein baseline_balance() mein dekha tha (do independent groups ki raw distributions compare karna, bina kisi regression-formula ke) — yahan ek doosra, non-parametric cross-check ke roop mein use ho raha hai, agar regression ka result kisi outlier ya distribution-shape ki wajah se misleading ho toh. Syntax: f"{outcome_col} ~ treatment + C(district)" — R-style formula string, f-string ke saath banayi gayi taaki outcome_col (jo "ndbi_placebo_change" ya "lights_placebo_change" ho sakta hai) dynamically

formula mein daala ja sake. `d.treatment == 1` — boolean condition, treated rows select karne ke liye; `d[condition][col]` — pehle rows filter karo, fir us column ki values nikaalo. `model.conf_int().loc["treatment"]` — confidence interval ki table se sirf treatment row nikaalna, PDF 12 se familiar pattern. Dictionary return karte waqt `(d.treatment==1).sum()` — boolean values ka sum, True ko 1 ki tarah gin kar total treated rows batata hai.

main() — Aur Ek Statistical-Honesty Ka Sabse Bada Sabak Is Poore Project Mein

```
def main():
    parser = argparse.ArgumentParser(description="Parallel-pre-trends placebo test, 2019 vs 2021 (Section 7.5).")
    parser.add_argument("--window", choices=["full_year", "summer"], required=True)
    args = parser.parse_args()
    window = args.window

    treated = build_group("treated", window)
    control = build_group("control", window)
    panel = pd.concat([treated, control], ignore_index=True)
```

Purpose: Bilkul familiar `argparse/build/concat` pattern — treated aur control, dono groups ka data banao, phir dono ko ek hi panel mein jod do (rows ki tarah, ek ke neeche ek). Syntax: `pd.concat([treated, control], ignore_index=True)` — PDF 12 mein `build_panel()` mein bhi ye dekha tha; `ignore_index=True` purani, alag-alag row-numbers ko bhula kar 0 se naya, continuous numbering deta hai.

```
results = {}
for outcome, col in [("ndbi", "ndbi_placebo_change"), ("lights",
"lights_placebo_change")]:
    r = placebo_did(panel, col)
    results[outcome] = r
    print(f"\n{outcome.upper()}: n={r['n_obs']} (treated={r['n_treated']},
control={r['n_control']})")
    print(f" Placebo DiD coef={r['placebo_did_coef']:+.5f} SE={r['placebo_did_se']:.5f} "
          f"95% CI [{r['placebo_did_ci_lo']:+.5f}, {r['placebo_did_ci_hi']:+.5f}]
p={r['placebo_did_p']:.5f}")
    print(f" Mann-Whitney U (raw distributions) p={r['mann_whitney_p']:.5f}")
    verdict = "PASSES (null, as hoped)" if r["placebo_did_p"] > 0.05 else "FAILS
(significant -- pre-trends diverge)"
    print(f" -> Parallel-pre-trends check: {verdict}")
```

Purpose: Loop dono outcomes (ndbi, lights) ke liye `placebo_did()` chalata hai, results dictionary mein store karta hai, aur numbers print karta hai. Aakhri line sabse zyada dhyaan dene wali hai: ek verdict variable jo sirf '`p > 0.05`' check karke seedha PASSES ya FAILS bol deta hai — ek bilkul mechanical, bina kisi nuance ke, single-threshold decision. Is function ka apna print-output isliye is baat ko capture nahi karta ki ek

result 'borderline' bhi ho sakta hai — agla EXAMPLE box dikhaayega ki jab ye code summer NDBI ke liye chala, tab kya hua, aur usse kaise handle kiya gaya (script ke apne verdict se alag, zyada honest tareeke se). Syntax: dictionary ko loop mein populate karna: `results[outcome] = r`. List of tuples par loop: `for outcome, col in [...], (...):` — do values ek saath unpack ho rahi hain har iteration mein. Conditional expression (ternary): "A" if condition else "B" — ek line mein if/else, seedha ek value assign karne ke liye, poora if/else block likhe bina.

EXAMPLE — Jab Ek Automatic Threshold Khud Galat Kahani Sunata — Entry 30 Ka Summer NDBI

Jab ye script pehli baar chali (Development Log Entry 30, us waqt ki 732-village control list par), teen combinations bilkul saaf, clean aaye: full-year NDBI (coef +0.00196, p=0.864), full-year lights (-0.00665, p=0.755), summer lights (+0.00047, p=0.968) — sabme koi significant farak nahi, exactly jo hum chahte the. Lekin summer NDBI mein: district-fixed-effects placebo DiD coefficient +0.01287 tha, p=0.06429 — jo upar wale verdict function ke hisaab se, chunki $0.06429 > 0.05$ hai, seedha 'PASSES' print kar deta. Lekin Development Log mein likhne wale ne (yaani is study ke researcher ne) is number ko us bare check par chhod kar aage nahi badh gaye — unhone khud likha: 'it does not get rounded up to a clean pass here just because the script's own auto-verdict function... printed "PASSES." 0.06429 conventional 0.05 threshold se upar hai zaroor, lekin bahut kareeb hai — 'borderline' — aur isi combination ki raw, unadjusted Mann-Whitney comparison sharply significant thi, p=0.00001. In dono numbers ko saath padhkar ek honest verdict likha gaya: district-composition zyada-tar, lekin poora nahi, ek real pre-existing farak ko absorb kar raha tha summer NDBI mein treated aur control villages ke beech.

EXAMPLE — Ye Ek Real Problem Kyun Nahi Hai — Direction Check

Sirf 'borderline hai' keh dena kaafi nahi hai — is finding ka asli DiD result (jo H4 mein already null hai) par kya asar padta hai, ye check karna zaroori tha. Entry 30 ne ise seedha check kiya: agar summer NDBI mein treated villages 2019-2021 ke placebo period mein already thoda upar ki taraf trend kar rahe the (jo is borderline result batata hai, coefficient positive hai), toh kya yahi pre-existing trend real, 2021-2025 wale summer NDBI DiD result ko bhi explain kar sakta hai? Jawaab: nahi, seedhe tareeke se toh bilkul nahi — kyunki real summer NDBI DiD ka sign negative hai (control-group DiD coefficient -0.00275, jo PDF 12/14 mein bhi dekha), jabki ye placebo-period effect positive hai (treated upar ki taraf trend kar raha). Agar pre-existing trend hi asli result ko explain karta, toh dono ka sign same hona chahiye tha — 'treated pehle se hi upar ja raha tha, isliye baad mein bhi upar dikha' jaisi seedhi kahani. Yahan aisa nahi hai — dono opposite directions mein hain, isliye ek simple 'ye pre-trend hi sab kuch explain kar deta hai' wali kahani fit nahi baithti. Result abhi bhi ek honestly-disclosed exception hai, ek chhupaya hua confound nahi.

Entry 37 Ka Fix, Aur Entry 42 Ka Rerun — Numbers Thode Shift, Kahani Wahi

EXAMPLE — Control-Join Fix Ke Baad, Poori File 721-Village List Par Dobara Chali (Entry 42)

PDF 14 mein humne Entry 37 ka district-verification fix dekha tha, jisne control list ko 732 se 721 villages tak sudhaara. Us fix ke saath-saath, isi file ka control-side join bhi village_id se coordinate-based join mein badal diya gaya tha (upar dekha). Lekin sirf code fix karna kaafi nahi tha — asli 2019 baseline data bhi 721-village list ke against dobara extract karna zaroori tha, jo Entry 42 mein hua: `extract_pretreatment_baseline.py --group control` aur `pretreatment_placebo_test.py`, dono windows ke liye, live Earth Engine access ke saath, apne machine par chalaayi gayi. Result: 'no reversal' — jaisa expect kiya gaya tha, kyunki 721-village list 732-village list ka hi ek subset hai (sirf genuinely galat, out-of-district villages hataaye gaye, koi naya, alag tarah ka sample nahi jodaa gaya), isliye numbers thode hi shift hue, koi conclusion nahi palta. Full-year NDBI: coefficient +0.00398 (pehle +0.00196), $p=0.73497$ (pehle 0.864) — clean. Full-year lights: -0.00647 (pehle -0.00665), $p=0.75898$ (pehle 0.755) — clean. Summer lights: +0.00632 (pehle +0.00047), $p=0.52021$ (pehle 0.968) — clean. Summer NDBI, wahi flagged exception, wahi flagged reh gaya: +0.01173 (pehle +0.01287), $p=0.07421$ (pehle 0.06429) — abhi bhi borderline, ab bhi clean pass mein round nahi kiya gaya — Mann-Whitney bhi abhi bhi sharply significant, $p=0.00003$ (pehle 0.00001). Ye is poore project ka ek achha example hai ki 'data-quality fix karne ke baad, kya hoga' predict karna kabhi bhi guess-work nahi hona chahiye — yahan predict kiya gaya tha ki kuch palte ga nahi, kyunki 721-list sirf 732-list ka subset hai, aur dobara chalaane ke baad wahi confirm hua.

Final State — pretreatment_placebo_test.py Ka Current, 721-Village Verdict

outputs/pretreatment_placebo_summary_fullyear.json aur
outputs/pretreatment_placebo_summary_summer.json se seedha liya gaya, current numbers:

Window / Outcome	Placebo DiD Coef	SE	p-value	Mann-Whitney p	Verdict
Full-year NDBI	+0.00398	0.01175	0.73497	0.78922	Clean pass
Full-year Lights	-0.00647	0.02109	0.75898	0.22102	Clean pass

Summer NDBI	+0.01173	0.00657	0.07421	0.00003	Borderline (flagged)
Summer Lights	+0.00632	0.00984	0.52021	0.66268	Clean pass

Teen combinations mein se teen, poori tarah null — jo is study ki parallel-trends assumption ke liye genuine, direct evidence hai. Chautha, summer NDBI, ek honestly-disclosed borderline exception hai — na toh chhupaya gaya, na overstate kiya gaya. Aur, jaisa upar dekha, iska direction khud is baat ka saboot hai ki ye asli, already-null summer NDBI DiD result ko explain nahi karta — dono opposite directions mein hain.

Part B — dw_pretreatment_placebo_test.py

Ye file bilkul wahi test hai jo humne abhi Part A mein padha, bas ek naye proxy — Dynamic World — ke liye. Dynamic World Google ka ek satellite-based land-cover classifier hai, jo NDBI ke seedhe do-band formula (PDF 11 mein seekha) ki jagah, ek machine-learning model use karta hai ye predict karne ke liye ki har satellite-pixel 'built-up' hai ya nahi. Iski poori triangulation-story — kyun ye interesting hai, NDBI/SAR se kaise disagree karta hai — PDF 18 mein aayegi. Yahan sirf itna jaanna kaafi hai ki Dynamic World ka apna 2021-2025 control-group DiD result significant nikla tha (jabki NDBI/SAR dono null the), aur ye file poochti hai: 'kya wo significant result sirf ek pehle-se-maujood trend hai, jo treatment se pehle se hi wahaan tha?'

Docstring — Is Baar, Motivation Khud Zyada Specific Hai

```
"""
```

```
Dynamic World parallel-pre-trends placebo test -- the DW analogue of
pretreatment_placebo_test.py. Checks 2019-to-2021 change in DW's "built"
probability, treated vs control, both pre-treatment (VVP-I wasn't sanctioned
till Feb 2023). Same motivation as the NDBI/lights placebo test, but run on
DW specifically: Section 6's triangulation already shows DW disagreeing with
NDBI on the control-group DiD (DW: small but significant effect; NDBI: null).
This test asks whether that DW gap was already there before treatment could
possibly apply -- if the 2019-to-2021 DW change is itself null, the later DW
control-group effect is not just a pre-existing DW drift picked up by chance.
```

```
Same spec as pretreatment_placebo_test.py's placebo_did(): district FE,
cluster-robust SE by district, one outcome (built) instead of two (ndbi,
lights), since Dynamic World only has the one "built" band.
```

```
Needs extract_dw_pretreatment_baseline.py run for both groups/windows first.
```

```
"""
```

```
import argparse
```

```
import json
```

```
import pandas as pd
import statsmodels.formula.api as smf
from scipy import stats
```

Purpose: Ye docstring Part A ke docstring se bilkul milta-julta structure follow karta hai, lekin ek zaroori extra context jodta hai: iska poora point sirf 'generic parallel-trends check' nahi hai — ye ek specific, targeted sawaal poochne ke liye likha gaya hai, kyunki Dynamic World ka control-group DiD result (jo PDF 18 mein poora aayega) NDBI/SAR se alag, significant nikla tha. Agar ye placebo test bhi significant aata (matlab treated aur control mein pehle se hi 2019-2021 mein Dynamic World ke "built" score mein farak tha), toh ye ek simple, boring explanation ban jaata us significant result ka — 'ye toh treatment se pehle se hi wahaan tha.' Is test ka null aana isliye ek meaningful, specific finding hai, na ki sirf ek formality. Syntax: Import statements Part A se bilkul identical hain — same libraries, same kaam ke liye.

Paths — Sirf Do Dictionaries, Ek Naya Comment

```
DW_PRETREATMENT_PATHS = {
    ("treated", "full_year"):
        "data/processed/border_optics_treated_dw_pretreatment_fullyear.csv",
    ("treated", "summer"):
        "data/processed/border_optics_treated_dw_pretreatment_summer.csv",
    ("control", "full_year"):
        "data/processed/border_optics_control_dw_pretreatment_fullyear.csv",
    ("control", "summer"):
        "data/processed/border_optics_control_dw_pretreatment_summer.csv",
}
# the "2021" side of this comparison is the built_before column already
# extracted by extract_dynamicworld_built.py (its full_year "before" window is
# 2021-01-01 to 2022-01-01, matching the NDBI/lights "before" definition)
EXISTING_DW_2021_PATHS = {
    ("treated", "full_year"):
        "data/processed/border_optics_treated_dynamicworld_fullyear.csv",
    ("treated", "summer"):
        "data/processed/border_optics_treated_dynamicworld_summer.csv",
    ("control", "full_year"):
        "data/processed/border_optics_control_dynamicworld_fullyear.csv",
    ("control", "summer"):
        "data/processed/border_optics_control_dynamicworld_summer.csv",
}
SUMMARY_OUT = {
    "full_year": "outputs/dw_pretreatment_placebo_summary_fullyear.json",
    "summer": "outputs/dw_pretreatment_placebo_summary_summer.json",
}
```

Purpose: Bilkul Part A jaisa hi structure — DW_PRETREATMENT_PATHS 2019 ka Dynamic World data batata hai, EXISTING_DW_2021_PATHS 2021 ka (already-extracted) data. Comment ek zaroori technical detail explain karta hai — ki 2021 ka 'before' window (extract_dynamicworld_built.py ne jo already banaya tha, PDF 18 mein poora dekhenge) exactly 2021-01-01 se 2022-01-01 tak hai, jo NDBI/lights ke apne 'before' definition se match karta hai — taaki dono placebo tests (NDBI/lights wala aur Dynamic World wala) ek hi time-window compare kar rahe hon, apples-to-apples. **Syntax:** Bilkul Part A jaisi dictionary-of-tuples-key structure, sirf naye file naam ke saath.

build_group() Aur placebo_did() — Same Pattern, Ek Outcome Kam

```
def build_group(group, window):
    """Returns one row per village: village_id, district, treatment,
    built_2019, built_2021, built_placebo_change (=2021-2019)."""
    baseline = pd.read_csv(DW_PRETREATMENT_PATHS[(group, window)])
    existing = pd.read_csv(EXISTING_DW_2021_PATHS[(group, window)])
    if group == "treated":
        existing = existing[existing["is_core_sample"] == True]
        baseline = baseline[["village_id", "built_2019"]]
        existing = existing[["village_id", "district", "built_before"]].rename(
            columns={"built_before": "built_2021"}
        )
        merged = existing.merge(baseline, on="village_id", how="inner")
    else:
        # control village_id gets reassigned fresh every time -- same issue
        # found and fixed in pretreatment_placebo_test.py -- join on rounded
        # coordinates instead, which are stable per village.
        baseline = baseline.copy()
        existing = existing.copy()
        baseline["_coord_key"] = list(zip(baseline["latitude"].round(6),
        baseline["longitude"].round(6)))
        existing["_coord_key"] = list(zip(existing["latitude"].round(6),
        existing["longitude"].round(6)))
        baseline = baseline[["_coord_key", "built_2019"]]
        existing = existing[["_coord_key", "village_id", "district", "built_before"]].rename(
            columns={"built_before": "built_2021"}
        )
        merged = existing.merge(baseline, on="_coord_key",
        how="inner").drop(columns="_coord_key")
        n_dropped = len(existing) - len(merged)
        if n_dropped:
            print(f" NOTE: {n_dropped} control village(s)... 2019 DW baseline is "
            f"for a different control-list vintage...")
        merged["treatment"] = 1 if group == "treated" else 0
        merged["built_placebo_change"] = merged["built_2021"] - merged["built_2019"]
    return merged
```

Purpose: Ye function line-by-line Part A ke `build_group()` ka hi copy hai, sirf do farak ke saath: (a) sirf ek outcome column hai — `built` — do (ndbi, lights) ki jagah, kyunki Dynamic World ka sirf ek hi "built" score hota hai, alag NDBI-jaisa ya lights-jaisa band nahi; (b) comment mein khud likha hai 'same issue found and fixed in `pretreatment_placebo_test.py`' — matlab ye file khud confirm karti hai ki coordinate-based join ka pattern yahan bhi wahi wajah se, wahi tareeke se apply kiya gaya, kisi doosri socch se nahi. Ye ek achha software-engineering habit hai — jab ek fix kisi file mein kaam kar chuka ho, doosri similar file mein bhi wahi fix consistently apply karna, aur ye baat comment mein explicitly likh dena ki dono jagah same reasoning hai. **Syntax:** Poora syntax Part A jaisa hi — dictionary key-lookup, boolean filter, column-select, rename, coordinate-key merge, sab wahi.

```
def placebo_did(panel, outcome_col):
    # same spec as pretreatment_placebo_test.py's placebo_did() -- outcome is
    # already the 2019-2021 change, so no 'post' term, just treated vs control
    d = panel.dropna(subset=[outcome_col]).copy()
    model = smf.ols(f"{outcome_col} ~ treatment + C(district)", data=d).fit(
        cov_type="cluster", cov_kwds={"groups": d["district"]}
    )
    coef, se, p = model.params["treatment"], model.bse["treatment"],
model.pvalues["treatment"]
    ci = model.conf_int().loc["treatment"]
    u_stat, mw_p = stats.mannwhitneyu(
        d[d.treatment == 1][outcome_col], d[d.treatment == 0][outcome_col]
    )
    return {
        "n_obs": len(d), "n_treated": int((d.treatment == 1).sum()),
        "n_control": int((d.treatment == 0).sum()),
        "placebo_did_coef": float(coef), "placebo_did_se": float(se), "placebo_did_p":
float(p),
        "placebo_did_ci_lo": float(ci[0]), "placebo_did_ci_hi": float(ci[1]),
        "mann_whitney_p": float(mw_p),
    }
```

Purpose: Ye function Part A ke `placebo_did()` se character-by-character identical hai — ek naye file mein bhi wahi formula, wahi cluster-robust SE, wahi Mann-Whitney cross-check, kyunki test ka logic khud outcome-agnostic hai (chahe outcome NDBI ho, lights ho, ya built) — ek hi statistical machinery, alag-alag column diya jaana chahiye. **Syntax:** Koi naya syntax nahi — poora function upar Part A mein already explain ho chuka hai.

main() — Ek Outcome, Isliye Loop Nahi, Seedha Ek Call

```
def main():
    parser = argparse.ArgumentParser(description="Dynamic World parallel-pre-trends
placebo test, 2019 vs 2021.")
    parser.add_argument("--window", choices=["full_year", "summer"], required=True)
```

```

args = parser.parse_args()
window = args.window

treated = build_group("treated", window)
control = build_group("control", window)
panel = pd.concat([treated, control], ignore_index=True)

r = placebo_did(panel, "built_placebo_change")
print(f"\nBUILT: n={r['n_obs']} (treated={r['n_treated']}, control={r['n_control']})")
print(f" Placebo DiD coef={r['placebo_did_coef']:+.5f} SE={r['placebo_did_se']:.5f} "
      f"95% CI [{r['placebo_did_ci_lo']:+.5f}, {r['placebo_did_ci_hi']:+.5f}]
p={r['placebo_did_p']:.5f}")
print(f" Mann-Whitney U (raw distributions) p={r['mann_whitney_p']:.5f}")
verdict = "PASSES (null, as hoped)" if r["placebo_did_p"] > 0.05 else "FAILS (significant
-- pre-trends diverge)"
print(f" -> Parallel-pre-trends check: {verdict}")

with open(SUMMARY_OUT[window], "w") as f:
    json.dump({"built": r}, f, indent=2)
print(f"\nSaved {SUMMARY_OUT[window]}")

```

Purpose: Part A ka main() ek for-loop use karta tha (do outcomes ke liye), yahan sirf ek outcome ("built_placebo_change") hai, isliye seedha ek baar placebo_did() call kiya jaata hai, koi loop nahi chahiye — ek chhota, sensible simplification jab sirf ek hi cheez test karni ho. Baaki poora structure — print statements, verdict logic, JSON save — Part A jaisa hi hai, bas is baar JSON mein {"built": r} — ek single-key dictionary, do outcomes ki jagah. Syntax: {"built": r} — ek dictionary literal jismein sirf ek hi key-value pair hai; json.dump() ise seedha file mein likh deta hai, wahi function jo is poore series mein baar-baar dekha gaya.

EXAMPLE — Entry 43 — Is Poore Project Ki History Mein Ye Test Pehli Baar Chala

Development Log ki Entry 37 se leke Entry 42 tak, is file ka status hamesha ek hi rehta tha: 'scripts ready, but not yet run' — extraction script (extract_dw_pretreatment_baseline.py) aur ye test dono likhi ja chuki thi, lekin live Earth Engine access na hone ki wajah se sandbox environment mein chalayi nahi ja sakti thi. Entry 43 mein ye finally chala — extract_dw_pretreatment_baseline.py dono groups aur windows ke liye (treated full-year 253/258 valid; treated summer 210/258; control full-year 716/721; control summer 614/721), fir dw_pretreatment_placebo_test.py dono windows ke liye, sab kuch current, 721-village district-verification-corrected control list ke against. Result: dono windows mein clean — full-year coefficient +0.00009, p=0.93406; summer coefficient +0.00207, p=0.17869. NDBI/lights wale placebo test ke ulta, yahan koi bhi borderline exception nahi mili — Dynamic World mein treated aur control villages 2019-2021 mein bilkul similar tarah se badle, dono windows mein.

EXAMPLE — Ye Result Kya Resolve Karta Hai, Aur Kya Nahi Karta

Ye finding genuinely useful hai, lekin isko overstate karna galat hoga — isliye Development Log khud dono baatein saath likhi hain. Jo ye resolve karta hai: Dynamic World ka 2021-2025 control-group DiD result significant kyun hai, uske liye ek specific, candidate explanation ko ruled out karta hai — ki 'ye sirf ek pehle-se-maujood trend hai jo treatment se pehle se hi wahaan tha.' Agar 2019-2021 mein bhi Dynamic World significant dikhta, toh ye ek strong reason hota us baad wale result ko shaq ki nazar se dekhne ka. Jo ye resolve NAHI karta: SAR aur NDBI dono null hain, jabki Dynamic World significant hai — is teeno proxies ke beech ka disagreement (jiski poori kahani PDF 18 mein aayegi) abhi bhi utna hi khula sawaal hai jitna pehle tha. Ye placebo test sirf ek specific, narrow sawaal ka jawaab deta hai — 'kya ye ek purana trend hai?' — 'kya Dynamic World sahi hai ya galat hai NDBI/SAR ke muqable mein?' jaisa bada sawaal alag hai, aur khula hi reh jaata hai.

Final State — dw_pretreatment_placebo_test.py Ka Current Verdict

outputs/dw_pretreatment_placebo_summary_fullyear.json aur
outputs/dw_pretreatment_placebo_summary_summer.json se seedha:

Window	n (treated / control)	Placebo DiD Coef	p-value	Mann-Whitney p	Verdict
Full-year	247 / 715	+0.00009	0.93406	0.65024	Clean pass
Summer	161 / 491	+0.00207	0.17869	0.10256	Clean pass

Dono windows clean — Dynamic World mein koi pre-existing, treatment-se-pehle wala divergence nahi mila. Yaad rahe ki yahan sample size chhota hai NDBI/lights ke muqable (247/715 aur 161/491, poore 251/721 ki jagah) — kyunki Dynamic World ka data har village ke liye available nahi hai, khaaskar summer window mein (jo already-dekha gaya pattern hai, PDF 12 mein bhi Dynamic World ke summer-window coverage-gap ka zikar hua tha).

Poori PDF Ka Synthesis — Teeno Placebo Tests Ek Saath

Is PDF mein humne kul 6 placebo-test combinations dekhe — NDBI aur lights, dono windows (4 combinations, pretreatment_placebo_test.py se), aur Dynamic World, dono windows (2 combinations, dw_pretreatment_placebo_test.py se). Inmein se 5 poori tarah clean hain, aur 1 (summer NDBI) ek honestly-disclosed, direction-checked borderline exception hai jo asli DiD result ko explain nahi karta. Ye poore project ke

parallel-trends assumption ke liye ek genuinely strong, multi-angle evidence hai — sirf ek proxy nahi, teen proxies; sirf ek window nahi, dono windows; aur sirf ek statistical test nahi, DiD regression aur Mann-Whitney, dono, ek hi jagah agree karte hain.

Is PDF mein humne dekha: DiD ka core assumption — parallel trends — seedha test nahi ho sakta, lekin 2019-2021 (VVP-I se pehle ka period) mein wahi test chala kar indirect evidence mil sakta hai. `pretreatment_placebo_test.py` ne NDBI aur lights ke liye ye test kiya, `did_model.py` jaisa hi formula use karke, bas 'post'/did_term term ke bina (outcome pehle se hi ek differenced change tha). Control-side join village_id ki jagah coordinate-based rakhi gayi — wahi Entry 37 wala fix jo PDF 14 mein bhi dekha tha. Teen combinations clean nikle, ek (summer NDBI) borderline reha ($p=0.07421$) — script ke apne bare $p>0.05$ verdict-check se alag, isse honestly ek exception ki tarah flag kiya gaya, aur direction-check se confirm kiya gaya ki ye asli result ko explain nahi karta. `dw_pretreatment_placebo_test.py` ne wahi test Dynamic World ke liye kiya (pehli baar Entry 43 mein chalaya gaya) — dono windows clean, ek specific candidate-explanation ko ruled out karte hue (ki DW ka significant control-group effect ek purana trend hai), bina SAR/NDBI-vs-DW disagreement ko resolve kiye.

Agla PDF (PDF 16) multiple-testing aur spatial correction dekhega — `holm_correction.py` (jab ek saath kayi tests chalaayein, toh false-positive risk kaise adjust karein) aur `spatial_moran_and_h3_correction.py` (jo PDF 13 mein H3 ke spatially-corrected p-values, 0.02949 aur 0.00350, ka poora computation dikhaayega). Jab bhi tayyar ho, bata dena.